

LUNELIS

ENGINE EXPLANATION

Duplicate Detection Engine

From first scan to safe quarantine, with the real thresholds

PRODUCT	Lunelis 0.46.0
DATE	2026-10-09
STATUS	Draft
AUDIENCE	Maintainers and curious power users
BASED ON	src/lunelis/dupes/ (hashing, detect, similar, quarantine, manage), ui/dupes_view.py, ui/near_view.py, jobs/engine.py, settings.py, CLAUDE.md

Statements in this document are read from the code and the changelog. Anything that could not be confirmed is listed under To check.

Contents

The problem it solves.....	3
Words used in this document.....	3
How it works.....	4
Where each stage runs.....	4
Exact copies.....	5
Step 1: choose candidates, folder by folder.....	5
Step 2: the sample hash.....	6
Step 3: likely groups.....	6
Step 4: verify.....	6
Step 5: choose the keeper.....	6
On the page.....	7
Group states on the page.....	7
The jobs engine in one minute.....	7
Near-duplicates.....	8
Step 1: the fingerprint.....	8
Step 2: find candidates without comparing everything.....	8
Step 3: a pair is one photo only if nothing contradicts it.....	8
Step 4: groups where everybody matches everybody.....	9
Step 5: the keeper and what is suggested.....	10
When it runs, and on the page.....	10
Setting aside, restoring and emptying.....	11
Setting aside.....	11
The life of a copy.....	12
Thresholds, settings and what it writes.....	13
Decision points.....	13
Settings.....	13
What it writes.....	14
How it keeps your files safe.....	15
A worked example.....	16
Exact track.....	16
Near-duplicate track.....	16
Known limits.....	17
Discrepancies.....	18
To check.....	19

CHAPTER 1

The problem it solves

WHERE Library > Find duplicates, and the Duplicates page	FOR Maintainers and power users with several photo folders	ANSWERS How are copies found, proven and set aside without loss?
--	--	--

The same photo ends up in many places: a backup folder, an old pool, a Google Takeout export, a resized copy sent to a friend. File names change, so names cannot find them. Reading every byte of a multi-terabyte library to compare it is far too slow, especially over a network share.

The engine works in two tracks. Each track only *proposes*. Nothing leaves your folders until you press a button, and even then it is renamed into a quarantine folder, never deleted. This document describes Lunelis 0.46.0.

Track	What it finds	How it knows	Code
Exact copies	Files with identical bytes	A cheap sample hash, then a full SHA-256	<code>dupes/detect.py</code> , <code>hashing.py</code>
Near-duplicates	The same picture saved as another file (resized, re-compressed, exported)	A 64-bit fingerprint of the thumbnail, then strict match rules	<code>dupes/similar.py</code>

192 KB most a sample hash reads from one file (3 slices of 64 KB)	~12x fewer candidates per folder once capture time joins file size as the key	4 bits most two 64-bit fingerprints may differ to count as one picture	1 place that removes files for good: Empty, on the Quarantine page
---	---	--	--

Words used in this document

Word	Meaning
Source	A folder tree Lunelis watches. The code calls it a root.
Keeper, extras	The copy that stays in a group, and the other members.
Sample hash	SHA-256 of the file size plus three 64 KB slices. A filter, never proof.
Full hash	SHA-256 of every byte, stored as <code>content_hash</code> . Proof of identical bytes.
Likely, Verified	Page labels for a <code>sampl</code> ed group and an <code>exact</code> (verified) group.
Fingerprint	A 64-bit difference hash (dHash) of a thumbnail: <code>files.perceptual_hash</code> .
Bits apart	How many of the 64 fingerprint bits differ (the Hamming distance).
Set aside	Rename into <code>_Lunelis Quarantine</code> on the same source. Reversible.

CHAPTER 2

How it works

The two tracks meet at the same last step. The exact track runs as background *jobs*, folder by folder. The near-duplicate track runs once thumbnails exist.

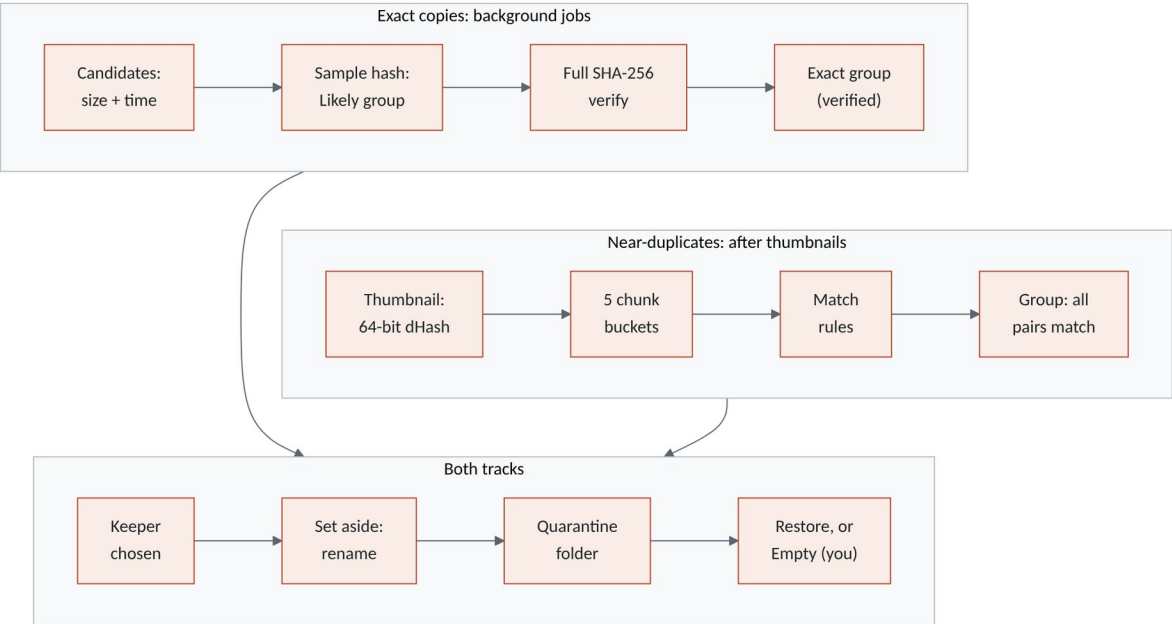


Figure 1. Overview: two tracks find and prove candidates; both end at one keeper choice and one reversible quarantine.

Where each stage runs

Stage	Started by	Runs as
Candidates, sample hash	Library > Find duplicates... (main_window.py:834)	Job duplicates , one folder at a time
Verify	"Verify all likely groups..." or "Verify this group"	Job verify ; the one-group button uses a worker thread
Fingerprint	Each thumbnail made (thumbnails.py:278)	Inside the thumbnail pass; older photos by compute_missing
Compare fingerprints	After each scan (main_window.py:238), or "Find again"	Library worker thread
Set aside	Buttons on the Duplicates page	Worker thread; one exact group runs on the interface thread

CHAPTER 3

Exact copies

Step 1: choose candidates, folder by folder

A folder counts as done only when its duplicates are known across the whole catalog, so results appear after the first folder, not after the whole library.

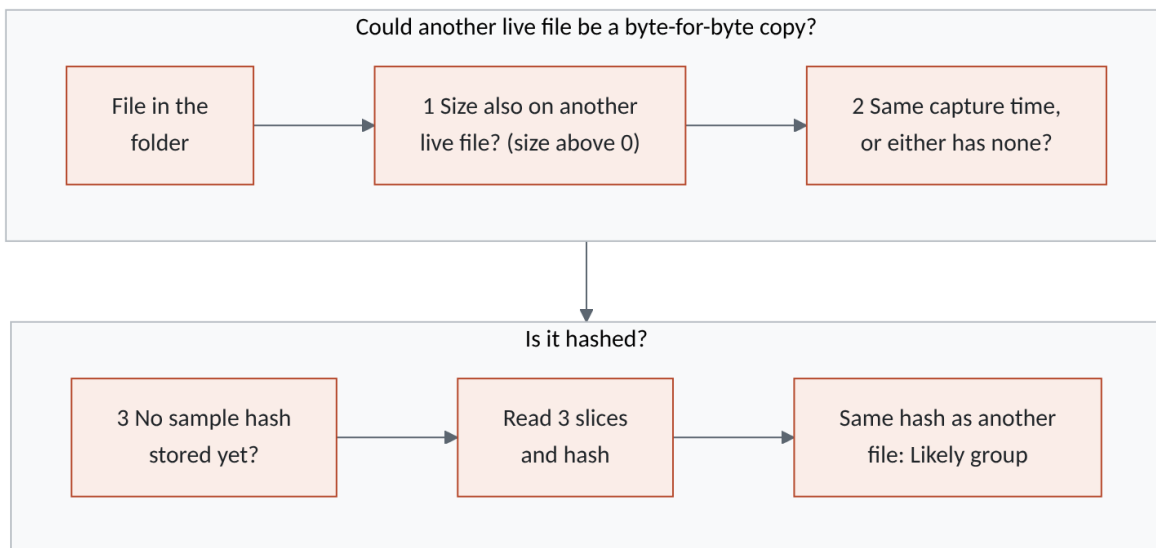


Figure 2. Candidate selection: a "no" at test 1 or 2 means the file is never read; a stored sample hash is reused, not recomputed.

The first test is a query for sizes in this folder that also occur on some other live file ([detect.py:157-161](#)), looked up 500 at a time ([detect.py:117](#)). The second compares capture times as stored text ([detect.py:131-137](#)), so "to the millisecond" means equal text.

Why capture time? Uncompressed Sony RAW files are all exactly the same size. With size as the only key, every RAW of one camera body was a candidate for every other, and a 15,841-file folder hashed 91,000 files in 424 s, reading 17.8 GB. Identical bytes imply identical EXIF, so real copies always share a capture time. A file with no capture time matches on size alone, so no true copy is missed.

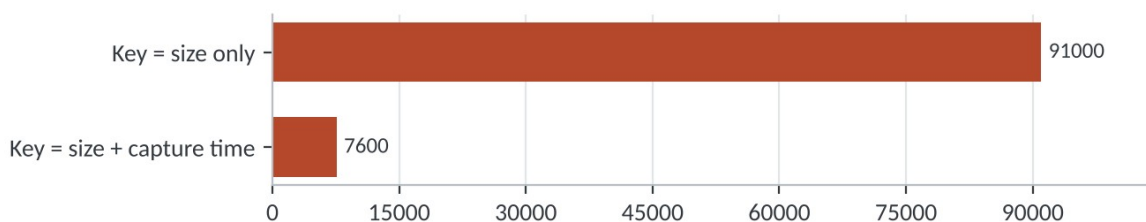


Figure 3. Files hashed for that folder (CLAUDE.md, Jobs and duplicates rules). The second bar is derived: 91,000 divided by the stated factor of about 12.

Step 2: the sample hash

For each candidate, one SHA-256 runs over the text "`<size>:`" and three 64 KB slices: the start, the middle (`size // 2 - 32 KB`) and the last 64 KB (`hashing.py:74-91`). A file of 192 KB or less is read whole. So a sample hash reads at most 196,608 bytes, against 20 to 40 MB for a full read of a typical photo (the range in the `hashing.py` docstring). Eight files are read at a time, sharing the speed limit. A file that cannot be read is listed with its error; a network error stops the folder and the job waits (`detect.py:85-90`).

Step 3: likely groups

Files that share a sample hash form a **sampl**ed group (`rebuild_groups, detect.py:178-202`); the page calls it **Likely**. A group with fewer than two live members is deleted. On the author's old-pool folder this gave 13,510 likely groups holding 689.7 GB of extra copies.

WARNING · A sample is only a filter

Two different files can share a size, a date and three slices. `quarantine()` refuses a sampled group outright (`quarantine.py:37-39`). Verify first.

Step 4: verify

`verify_group(detect.py:207-229)` reads **every byte** of every member in 1 MB pieces (`hashing.py:94-115`), four files at a time. A file with a stored `content_hash` is not read again. Members are regrouped by full hash into **exact** groups with `verified = 1`. If the sample was a coincidence, the full hashes differ and no exact group forms; the likely group stays in the catalog but is hidden once all its members have a full hash. A trigger sets `verified = 0` when a member's size or date changes (migration 41, `schema.py:877-882`).

Step 5: choose the keeper

Identical copies have identical content, so the keeper is chosen by *where* it sits. `keeper_rank(detect.py:234-246)` sorts by this tuple; the first wins.

Order	Test	Code
1	Position in your "keep the copy in" source	<code>preferred_roots</code>
2	Not a Takeout copy (word "takeout" in source path plus file path)	<code>detect.py:243</code>
3	Fewest folder levels, then shortest relative path	<code>rel.count("/"), len(rel)</code>
4	Lowest file id (cataloged first)	<code>fid</code>

Your own choice beats the rules: **Keep this one** stores `is_keeper = 1` on that file (`dupes_view.py:101-105`) and the page prefers the marked file (`dupes_view.py:95`).

On the page

The **Exact copies** tab lists verified groups plus likely groups not yet fully hashed, biggest saving first. A likely group whose every member already has a full hash is hidden, on the assumption that a verified group stands for it (`dupes_view.py:92`; see Discrepancies). The Show box filters All, Verified only, Likely only. A group's detail panel shows the kept copy, each extra (with **Show** and **Keep this one**), and one button: **Verify this group** or **Set aside N extra copies**.

Group states on the page

Status	What it means	Button	Set aside
Likely	Group sampl ed; some member has no full hash	Verify this group	Refused
Verified	Group exact ; every member hashed in full	Set aside N extra copies	Yes
Not listed	A sampled group whose members all have a full hash	none	n/a
Verified, flag cleared	Group exact but verified = 0 after a file changed (see Discrepancies)	Set aside N extra copies	Refused

The jobs engine in one minute

A job is a kind, some options and a list of folders (`jobs/engine.py:172-214`). The engine knows nothing about what a kind does to a folder; it keeps the bookkeeping that makes work safe to stop.

- **One folder at a time.** A folder is marked done only after its work is committed (`engine.py:329`), and every hash is committed as it is made (`detect.py:166-167`). A pause, quit or power cut loses at most the folder in progress; jobs left running are queued again at start-up (`engine.py:226`).
- **Offline shares wait.** A network error raises `SourceOffline`; the job switches to **waiting** and retries every 60 s (`engine.py:32,318-325`). The file is not marked bad (`hashing.py:38-43`).
- **Schedule and speed.** **now**, **idle** or a nightly **window** (`engine.py:256`). One shared MB/s cap covers all hashing threads (`Throttle`, `hashing.py:46-62`).
- **Threads.** Eight workers for the sample pass, at most four for full hashing (`engine.py:56,85`).
- **Scope.** Find duplicates asks for sources or one folder, when to run and a speed limit, with Settings as defaults (`ui/jobs.py:240-310`). Library > Hash everything (`main_window.py:835`) is a separate **full_hash** job that only builds an integrity baseline.

CHAPTER 4

Near-duplicates

Step 1: the fingerprint

A thumbnail is shrunk to 9 by 8 grey squares. Each of the 8 rows asks, 8 times, "is this square brighter than its right-hand neighbour?" That gives 64 bits, kept as 16 hex digits (`dhash`, `similar.py:53-61`). Pictures that look alike get nearly the same bits, whatever their size or JPEG quality.

- **Free for new photos:** taken from the picture in memory while the thumbnail is made (`thumbnails.py:278`).
- **Older photos:** fingerprinted from the cached 512 px thumbnail, never the original or the NAS, 2,000 at a time on 8 threads (`similar.py:64-94`).
- **Stale fingerprints are dropped:** the scanner clears `perceptual_hash` when a file's size or date changes (`scan.py:361`).

Step 2: find candidates without comparing everything

Comparing every pair of photos in a library of 159,000 files is out of the question. Each fingerprint is cut into five chunks of 13, 13, 13, 13 and 12 bits (`similar.py:97-103`). Two fingerprints that differ in at most 4 bits cannot disturb all five chunks, so at least one chunk is identical. Each photo is filed into five buckets; only photos in the same bucket are compared.

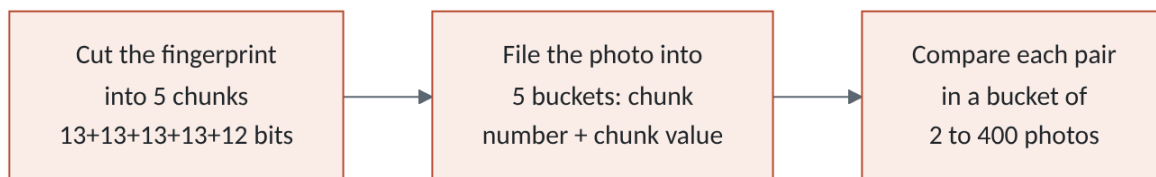


Figure 4. Candidate search: five buckets per photo; a bucket of more than 400 photos says nothing and is skipped.

Three filters apply first (`find_groups`, `similar.py:115-149`). **RAW files are excluded:** a RAW's copies are exact copies, and a RAW and its JPEG differ on purpose. **Near-blank fingerprints** (fewer than 8 or more than 56 bits set: black frames, flat sky) skip the chunk buckets. **Same-name buckets:** photos with one file name (2 to 8 of them, any folder) get their own bucket, so a copy whose picture moved further can still be compared.

Step 3: a pair is one photo only if nothing contradicts it

`same_photo` (`similar.py:216-242`) applies six gates in order. The first failure rejects the pair. A *shifted copy* means the same file name with capture times more than 60 s and up to 36 h apart.

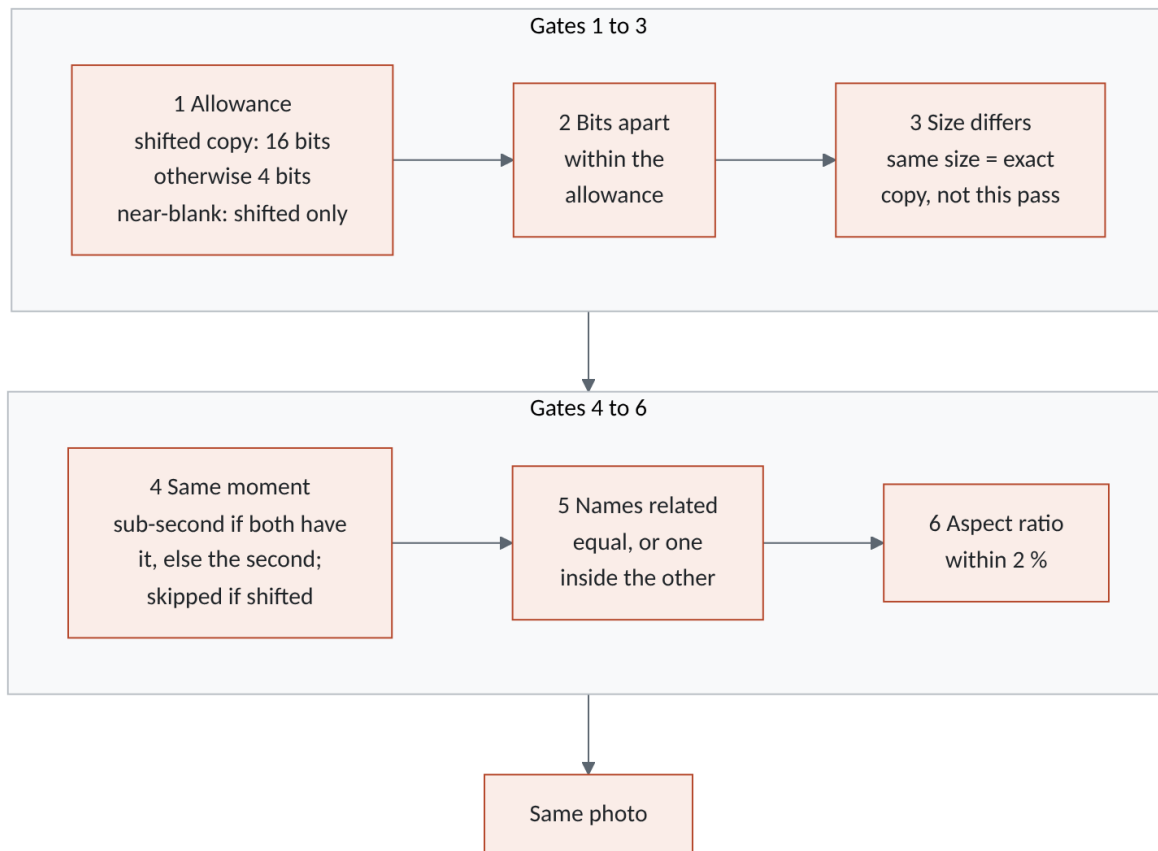


Figure 5. The near-duplicate decision: six gates, in code order, with the real thresholds. Any failed gate rejects the pair.

The reasons: burst frames 0.125 s apart look alike but are different shots (gate 4); copies keep their name, so unrelated names at one moment are another photo (gate 5); a crop is a different picture (gate 6). A missing capture time never contradicts. Aspect ratios are compared after EXIF orientation.

The shifted-copy exception. A Takeout copy of an edited photo can come back hours from the camera's clock, and edits move the picture further than a plain re-encode. So for a shifted copy the moment test is waived and up to **16 bits** are allowed (`EDITED_DISTANCE`, `similar.py:197-207,226`). A gap of a minute or less is a burst frame, never a shifted copy. A near-blank picture is allowed only as a shifted copy, and only within 4 bits.

Step 4: groups where everybody matches everybody

Matches are joined by a greedy complete-linkage pass (`similar.py:166-181`): seeds in order of most matches, then a candidate joins only if it matches **every** current member. Byte-identical files count as matching, so they do not keep an edit of the same shot out of the group. Chaining would be wrong: if A is 3 bits from B and B is 3 bits from C, A and C may be 6 bits apart and unrelated.

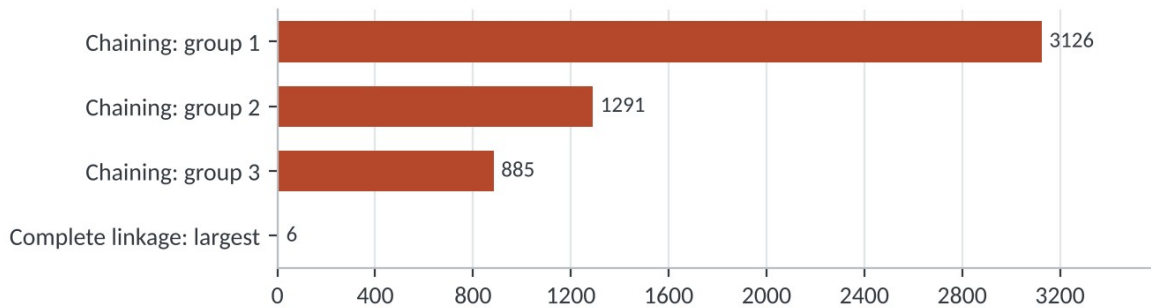


Figure 6. Largest near-duplicate group on the author's library (CLAUDE.md, Gotchas): chaining versus complete linkage.

Step 5: the keeper and what is suggested

`keeper_rank` (`similar.py:342-358`) sorts by this tuple; the first wins. It differs from the exact rule because near-duplicates differ in quality.

Order	Test
1	Not damaged (no row in damaged)
2	Not an edit (relative path contains "edit" or "export")
3	Most pixels (width times height)
4	Not in a Takeout source (by the source's own folder name)
5	Position in your "keep the copy in" source
6	Biggest file
7	Fewest folder levels, then lowest file id

Only a **plainly lesser** copy is pre-ticked (`suggest`, `similar.py:301-310`): not an edit, and fewer pixels than the keeper, or in a Takeout source, or with the keeper's file stem. A same-pixel copy under another name might be the original of an edit, so it stays unticked; you can still tick it.

When it runs, and on the page

`similar.refresh` (`similar.py:272-282`) fingerprints what is missing and regroups only if the count of fingerprinted live files differs from the stored `similar_grouped_count`. Regrouping replaces all `similar` groups at once (`rebuild`, line 251). "Find again" always regroups. On the **Near-duplicates** tab each copy shows **Kept**, **Edit - kept** (no tick box) or a "Set aside" tick; buttons set aside the ticked rows of one group, or every suggested copy, after a confirmation (`near_view.py:178-243`).

CHAPTER 5

Setting aside, restoring and emptying

Setting aside

Set aside is the only action the engine takes on your files. `quarantine()` (`quarantine.py:33-72`) handles exact groups; `quarantine_similar()` (`similar.py:361-393`) handles near-duplicates.

- 1. Check the request.** Exact: the group must be `exact` and verified, each file a live member, and one live copy must remain. Near: the group must be `similar` and one live member must remain.
- 2. Snapshot the catalog** into `<data>/backups` as `catalog-<time>-before-quarantine.zip` (`quarantine.py:51-53`). Bulk buttons snapshot once, before the first move.
- 3. Pick the copy that receives your work.** Exact: the marked keeper, else the lowest id left. Near: the best remaining copy by the keeper ranking.
- 4. Merge your work into it** (`merge_user_data, migrate/execute.py:155-178`; table below).
- 5. Exact only: carry the sidecar.** If the set-aside copy has an XMP sidecar and the keeper has none, a copy goes beside the keeper, never overwriting a file (`carry_sidecar`, line 181).
- 6. Move the pair.** Photo and sidecar are renamed into `<source>\_Lunelis Quarantine\<same relative path>` (`move_pair, quarantine.py:94-109`). On a name clash both get the file id, as in `IMG_0412 (5231).ARW`, decided before anything moves (`quarantine.py:75-91`).
- 7. Commit per file** (`quarantine.py:67-69`), so a crash cannot lose track of one.

The move is a rename inside one source: instant, and it needs no free space.

Moved to the keeper	Rule
Stars, pick flag, colour label	The keeper's own non-empty value wins; an empty one takes the other's. Marks the sidecar for rewriting
Albums	Added, keeping the album position
Tags	Manual tags only; suggested scene tags are not carried
Event	Only if the keeper is in none
Not moved	Edit stacks, face data, map pins: they stay with the set-aside row

Exact and near-duplicate set-aside compared

Point	Exact (<code>quarantine</code>)	Near (<code>quarantine_similar</code>)
Group must be	<code>exact</code> and verified	<code>similar</code>
Non-member ids	Refused with an error	Silently ignored
Receives your work	Marked keeper, else lowest id	Best remaining by keeper ranking
Sidecar carried to keeper	Yes, if the keeper has none	No
Snapshot	Before the first move, if a folder is given	The same

Set aside, in order

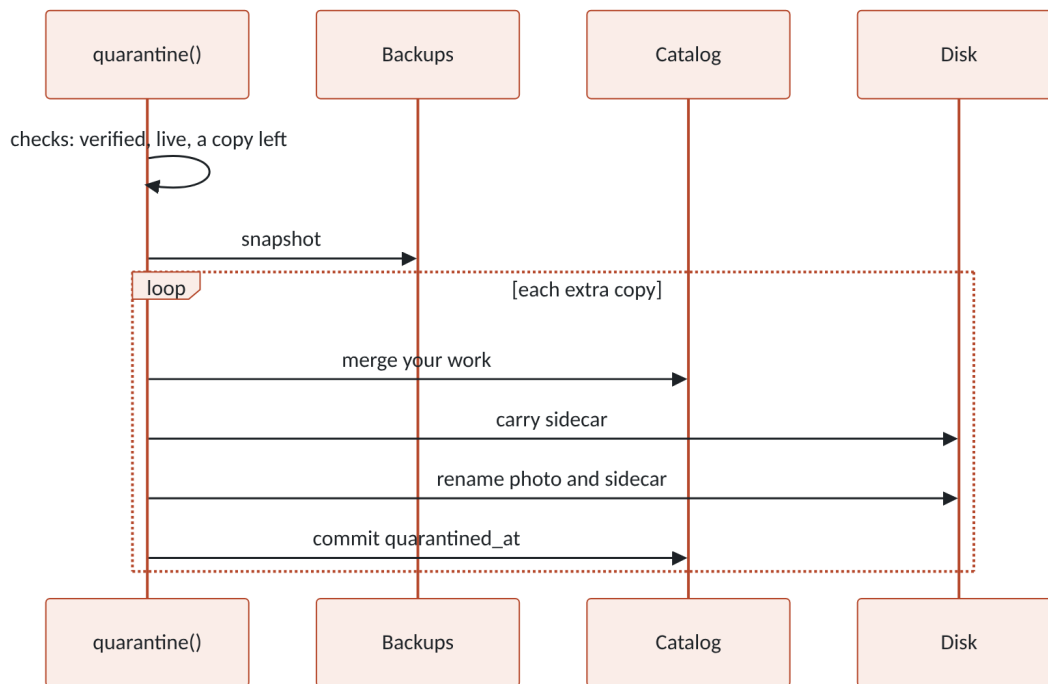


Figure 7. Set aside for an exact group: checks first, then one file at a time, committed before the next moves.

The life of a copy

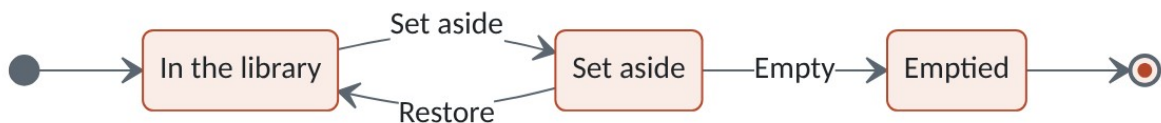


Figure 8. One copy's life: set aside is reversible; Empty is the only exit, and always your decision.

Restore (`quarantine.py:157-172`) renames photo and sidecar back, refusing if the old path is occupied, and clears `quarantined_at`. The scanner never calls a quarantined file missing and skips the `_lunelis quarantine` folder (`scan.py:33-36,327-329`).

Empty (Quarantine page, `manage.py:225-288`) demands for each file: it is still in quarantine; the kept copy exists (same size for byte copies); the kept copy is **byte-for-byte the same** (full SHA-256 of both, `manage.py:291-306`). Then the file goes to the Recycle Bin on a local NTFS or ReFS fixed drive, or is deleted for good elsewhere. A row is written to **purged**, the file's catalog row is deleted, its sidecars follow, and a **before-empty-quarantine** snapshot was taken first. The page labels each entry Exact copy, Near-duplicate, Migration copy or Migrated original, and works out its kept copy as the lowest-id live member of the file's group (`manage.py:30-31,115-131`). Only for exact copies does Empty compare bytes; for near-duplicates it checks that the kept file exists, because the copies differ by design. "Keep set-aside files" offers forever (default), 30, 90 or 365 days; after that files are only *offered* for removal (`manage.due`, line 134).

CHAPTER 6

Thresholds, settings and what it writes

Decision points

Question	Threshold	Where
Sample slices	3 x 64 KB; whole if 192 KB or less	hashing.py:20,79,84
Full-hash read size	1 MB	hashing.py:21
Candidate capture time	Equal text, or either has none	detect.py:131-137
Threads	8 sample pass, 4 full hash	detect.py:28,208
Fingerprint	64 bits, 9 x 8 grey	similar.py:55-61
Maximum bits apart	4	similar.py:47
Chunks per fingerprint	5 (13/13/13/13/12)	similar.py:100
Largest bucket compared	400 photos	similar.py:48,156
Near-blank fingerprint	Under 8 or over 56 bits set	similar.py:127,223
Same-name bucket	2 to 8 photos	similar.py:148
Shifted copy	Same name, 61 s to 36 h apart	similar.py:197,207
Bits for a shifted copy	16 (4 if near blank)	similar.py:198,226
Aspect ratio	Within 2 %	similar.py:240
Edit words	"edit" or "export" in the path	similar.py:248

Settings

Setting	Default	Range	Effect
preferred_roots	empty	List of source ids	Keeper = copy in first source
job_default_when	now	now, idle, window	Default "when to run"
job_idle_minutes	5	1 to 1,440	Idle: no input this long
job_window_start_hour, job_window_end_hour	22 and 6	0 to 23 each	Night window; may pass midnight
job_mb_per_s	0 (none)	0 to 100,000	Read-speed cap, MB/s
trash_keep_days	0 (forever)	0, 30, 90, 365	Age to offer for removal

Defined at `settings.py:32,45-49,88-89`, validated at `settings.py:146-169` (`trash_keep_days` has no range check; the page offers the four values). The internal counter `similar_grouped_count` records how many fingerprints the last grouping used. The thresholds above are constants.

What it writes

Where	What	When
<code>files.sample_hash</code>	Sample hash	Duplicates job; cleared if the file changes
<code>files.content_hash</code>	Full SHA-256, only if empty	Verify, Hash everything, integrity, backup
<code>files.perceptual_hash</code>	64-bit fingerprint	Thumbnail pass, <code>compute_missing</code>
<code>duplicate_groups, duplicate_group_files</code>	Groups <code>sampled</code> , <code>exact</code> , <code>similar</code> ; <code>verified</code> ; <code>is_keeper</code>	Each pass; similar rebuilt whole
<code>jobs, job_folders</code>	State, folders done, bytes read	While a job runs
<code>files.quarantined_at, quarantine_path</code>	Where and when set aside	Set aside; cleared by Restore
<code>ratings, album_files, file_tags, event_files</code>	Your work merged into the keeper	Set aside
Sidecar beside the keeper	Copy of the extra's XMP	Exact only, if the keeper has none
<code><source>_Lunelis Quarantine\...</code>	Renamed photo and sidecar	Set aside
<code><data>\backups\...before-quarantine.zip</code>	Catalog snapshot	Before the first move of a batch
<code>purged</code> , Recycle Bin or deletion	Record of what was emptied	Only when you empty

CHAPTER 7

How it keeps your files safe

Guarantee	Enforced by
A sample hash never acts alone	<code>quarantine()</code> refuses non-exact or unverified groups (<code>quarantine.py:37-39</code>)
The last live copy never moves	<code>quarantine.py:48-49</code> ; <code>similar.py:375-377</code>
Only live members of the named group move	<code>quarantine.py:45-47</code>
The catalog is snapshotted before moves	<code>quarantine.py:51-53</code> ; <code>similar.py:378-380</code>
Nothing is deleted: set aside is a same-source rename	<code>quarantine_paths</code> , <code>move_pair</code> (<code>quarantine.py:75-109</code>)
Photo and sidecar move together or not at all	<code>move_pair</code> checks both targets, undoes on failure (<code>quarantine.py:98-109</code>)
Nothing is overwritten; clashes settled first	Id suffix and refusal (<code>quarantine.py:82-101</code>)
A crash cannot lose track of a file	Commit per file (<code>quarantine.py:69</code>); folder-by-folder jobs
Your stars, albums, tags and event survive	<code>merge_user_data</code> before the move
A sidecar is not left only in quarantine	<code>carry_sidecar</code> (exact copies)
A cross-drive move is verified	<code>move_one</code> copies, compares, then removes (<code>quarantine.py:112-129</code>)
A changed file loses verified status and stale hashes	Trigger <code>files_changed_unverify</code> ; <code>scan.py:361</code>
An offline share pauses the job, marks nothing bad	<code>offline_error</code> (<code>hashing.py:38-43</code>); <code>engine.py:318-325</code>
Edits and exports are never suggested	<code>is_edit</code> , <code>suggest</code> (<code>similar.py:245,307</code>); no tick box (<code>near_view.py:199</code>)
RAW files never enter near-duplicate matching	<code>is_raw = 0</code> (<code>similar.py:121</code>)
Quarantined files are not "missing" or re-cataloged	<code>scan.py:33-36,327-329</code>
Restore never overwrites	<code>move_pair</code> refuses an occupied path
Emptying needs a present, byte-identical kept copy	<code>manage.empty</code> (<code>manage.py:225-288</code>)
Nothing is emptied automatically	<code>manage.due</code> only offers (<code>manage.py:134-151</code>)

CHAPTER 8

A worked example

All names and sizes are made up. Two sources are watched: `D:\Photos` and `E:\Backup Pool`.

- `D:\Photos\2024\6-19-2024 Air Show\IMG_0412.ARW`: about 61 MB, captured 10:22:31.250, rated 3 stars.
- `E:\Backup Pool\Air Show 2024\IMG_0412.ARW`: a byte-identical copy, rated 4 stars, in the album "Air Shows".
- `D:\Photos\2024\6-19-2024 Air Show\IMG_0413.ARW`: the same size, captured 10:22:31.375, another shot.
- `...\IMG_0420.JPG` (6000 x 4000, 14 MB) in the same folder; its Takeout copy `D:\Google Takeout 2024\Google Photos\Air Show\IMG_0420.JPG` (2048 x 1365, 1.1 MB, time 8 hours later); and an edit `D:\Photos\Exports\Air Show edit\IMG_0420-edit.JPG`.

Exact track

1. **Find duplicates** runs on both sources. In the folder `2024/6-19-2024 Air Show` the RAW size occurs elsewhere. `IMG_0413.ARW` shares the size but not the capture time, so it is **never read**. Only the two `IMG_0412.ARW` files are candidates.
2. Each gives a sample hash from three 64 KB slices: about 192 KB read instead of 61 MB. They match: a **Likely** group of two.
3. **Verify this group** reads both files fully. The full hashes match, an **exact** group forms and the status becomes **Verified**.
4. **Which copy stays?** E: is shallower (one folder level against two), so by default it would be the keeper. You choose "keep the copy in D:\Photos", which puts D: first in the order.
5. **Set aside 1 extra copy**. A snapshot is written. D: keeps its 3 stars and gains the album. The E: file is renamed to `E:\Backup Pool\_Lunelis Quarantine\Air Show 2024\IMG_0412.ARW`. Nothing was copied or deleted.

Near-duplicate track

1. The original and the edit are fingerprinted 2 bits apart, the Takeout copy 11 bits (illustrative). They share a bucket, so they are compared.
2. Original and Takeout copy: same name, times 8 h apart, so a **shifted copy** (16 bits allowed); sizes differ; aspect 1.5000 against 1.5004 is within 2 %. **Match**.
3. Original and edit: `img_0420` is inside `img_0420-edit`, 2 bits, same moment, other size. **Match**. All three match each other: one complete group.
4. **Keeper**: the original (an edit ranks lower; it has the most pixels). **Suggested**: the Takeout copy (fewer pixels, Takeout source). The edit shows **Edit - kept**.
5. After you confirm, the Takeout copy moves to `D:\Google Takeout 2024\_Lunelis Quarantine\Google Photos\Air Show\IMG_0420.JPG` and its ratings go to the original. Restore puts it back.

CHAPTER 9

Known limits

- **Exact copies are byte-identical only.** One changed byte makes a different file; re-saved JPEGs belong to the near-duplicate track.
- **Near-duplicates need a thumbnail first** and never cover RAW files. A RAW and its JPEG are never grouped.
- **A renamed copy with an unrelated name is not matched**, and neither is a cropped copy (the aspect gate).
- **A near-blank picture** matches only as a shifted copy, within 4 bits.
- **"edit" and "export" are plain substrings** of the relative path, so a folder named `Credits` counts as an edit and is never pre-ticked.
- **The same-name rule trusts the name.** The camera model is read but never compared (`similar.py:125-145`), so two cameras writing `IMG_0001.JPG` within 36 hours, under 16 bits apart, could be paired.
- **"Keep this one" is not durable.** `rebuild_groups` deletes and re-inserts a group's members, resetting `is_keeper` (`detect.py:193-199`).
- **Edit stacks and face data are not merged** into the keeper. Emptying deletes the catalog row and `edits` rows cascade (`schema.py:563`).
- **Near-duplicate set-aside does not carry the sidecar** (`quarantine_similar` never calls `carry_sidecar`).
- **The default keeper can be the backup.** "Fewest folder levels" may prefer a shallow backup folder over your organised library, as in the worked example. That is why "keep the copy in" exists.
- **Disabled sources and skipped folders are invisible** to both tracks: they filter on `excluded = 0` and `roots.enabled = 1` (`detect.py:31`, `similar.py:121`).
- **A photo without a thumbnail has no fingerprint**, so a failed thumbnail (`thumb_error`) keeps it out of near-duplicate matching.
- **Worst case per bucket is 79,800 pair tests** (400 choose 2) before the cap skips larger buckets.
- **A full verify reads every byte.** A large library takes hours, so it is a pausable job.

REFERENCE

Discrepancies

Where code and docstrings, CLAUDE.md or the CHANGELOG disagree, this document follows the code.

- **Candidate rule.** `detect.py:5-7` and the page text (`dupes_view.py:241`) say candidates share a size. The code also needs the same capture time or none (`detect.py:106-142`), as CLAUDE.md says.
- **Near-duplicate rules.** `similar.py:14-29` and CLAUDE.md list "same moment" and "near-blank skipped" as absolute. The code lets same-name copies 61 s to 36 h apart match at up to 16 bits without the moment test, and lets a near-blank picture pair as a shifted copy (`similar.py:197-242`).
- **Unused code and a missing step.** `TIME_TOLERANCE_S` (`similar.py:49`) and `_t()` (line 106) are never used. The `keeper_rank` docstring and the tab's note (`near_view.py:105-108`) omit the "is an edit" step the code ranks second (`similar.py:356`).
- **Takeout test differs.** The exact keeper counts "takeout" anywhere in source plus file path (`detect.py:243`); the near-duplicate track uses `takeout_roots`, the source's own folder name (`takeout.py:220-234`), as CLAUDE.md says is right.
- **"Exact name".** CLAUDE.md says a copy with the keeper's exact name is suggested. The code compares file stems without extension, lowercased (`similar.py:297`), so `IMG_1.png` matches `IMG_1.jpg`.
- **Quarantine location.** `move_one`'s docstring (`quarantine.py:112-117`) mentions a Lunelis-folder Duplicates and Trash and cross-drive copies. The duplicates code only builds paths under the file's own source; that branch serves migration.
- **"Network" in Empty.** CLAUDE.md and `manage.py` say network files are deleted, local ones recycled. `_is_network` means "has no Recycle Bin" (`paths.has_recycle_bin`): also USB sticks, FAT and exFAT volumes.
- **Verified label.** `load_groups` passes `method == "exact"` to `Group` (`dupes_view.py:96`) and ignores the `verified` column. After the migration 41 trigger clears it, the page still says Verified and offers Set aside, but `quarantine()` refuses, and the verify job handles only `sampled` groups (`engine.py:75`). CHANGELOG 0.37.3 says such a group "stops counting as verified". Trigger and refusal confirmed on a scratch catalog; the page was not run.
- **Hidden, not promoted.** The page hides a sampled group whose members all have a `content_hash` ("already represented by its verified group", `dupes_view.py:92`). Hash everything, integrity and backup runs also write full hashes; if one ran first, nothing creates the exact group, because `verify_duplicates` only looks at members with an empty `content_hash` (`main_window.py:3010-3018`). Read from code, not reproduced.
- **Verify ignores Settings.** `verify_duplicates` calls `create_job` with no options (`main_window.py:3017`): it runs now, with no speed limit, though the tooltip says it "can pause and run at night" (`dupes_view.py:265`). "Verify this group" has no throttle either.
- **Snapshot folder.** Both tabs snapshot into `paths.BACKUP_DIR` (`dupes_view.py:194,469`; `near_view.py:65`), not the folder `backup.backup_dir` returns for `catalog_backup_dir`.
- **Small ones.** `_verify_folder` says a fluke group is "dissolved"; the row stays, hidden. `quarantine()` raises on a non-member, `quarantine_similar()` silently ignores it (`similar.py:374`). The `trash_keep_days` comment sits on the `similar_grouped_count` line (`settings.py:88-89`).

REFERENCE

To check

- **Hash first, then duplicates.** In the real app, run Hash everything or a backup, then Find duplicates. Does the copy pair appear, and by what route does it become verified? (`dupes_view.py:92`, `main_window.py:3010`)
- **Verified label after a change.** Change a file in a verified group, rescan, open the page: does it say Verified, and does Set aside fail with the refusal?
- **"Keep this one" and Verify.** Mark a keeper on a likely group, verify, and see whether the mark is gone (`detect.py:193-199`).
- **Videos.** `find_groups` excludes only RAW; videos have poster thumbnails and fingerprints. Not tested whether video pairs group, or whether that is intended (`similar.py:121`).
- **Same-name false positives.** How often do two cameras give one file name within 36 hours at under 16 bits? Needs a run on a catalog copy opened with `immutable=1`.
- **Sidecars of near-duplicates.** Is skipping `carry_sidecar` intended? CHANGELOG 0.37.5 describes the fix for "the Duplicates page" only.
- **Edit stacks.** Confirm in the app that an edit on a set-aside copy is lost after Empty.
- **Quoted measurements** (424 s, 17.8 GB, 91,000 files, about 12x, 13,510 groups, 689.7 GB, group sizes 3,126, 1,291, 885 and 6) come from CLAUDE.md and were not re-measured. The 7,600 in the chart is 91,000 divided by 12, not a measurement.
- **Test suite.** pytest and PySide6 are not installed here, so `tests/test_dupes.py` and `tests/test_similar.py` were not run. `same_photo`, `shifted_copy`, `_chunks` and `quarantine()` were called directly instead.
- **Windows-only paths.** The Recycle Bin call, `GetDriveTypeW` checks and the network error codes (`hashing.py:24`) were read, not run.